

2026

REACT:

DO ZERO AO DASHBOARD

APOSTILA DO CURSO

Seu guia completo
para criar interfaces modernas
com React.

coursehub.com.br

Introdução ao React

Bem-vindo ao React

React é uma biblioteca JavaScript criada para desenvolver interfaces de usuário modernas, rápidas e reutilizáveis. Em vez de manipular diretamente o HTML da página, o React permite construir componentes independentes que podem ser combinados para formar aplicações completas.

Essa abordagem facilita a manutenção do código, melhora a organização do projeto e torna o desenvolvimento muito mais produtivo.

Hoje, React é utilizado por empresas como Meta, Netflix, Airbnb, Discord e milhares de startups ao redor do mundo.

Criado em 2013 pela Meta (Facebook), o React surgiu para resolver o desafio de construir interfaces complexas de forma organizada e eficiente. Seu principal diferencial é a arquitetura baseada em componentes reutilizáveis e a utilização do Virtual DOM para otimizar atualizações na interface.

Atualmente o React pode ser utilizado em diversos tipos de projetos:

- Dashboards administrativos
- Sistemas empresariais
- Plataformas EAD
- Redes sociais
- E-commerce
- Landing Pages
- Aplicativos Mobile (React Native)
- Interfaces Desktop (Electron)

Uma das maiores vantagens do React é seu enorme ecossistema de bibliotecas, permitindo construir aplicações completas utilizando ferramentas modernas.

O que você aprenderá neste curso

Ao concluir este curso você será capaz de:

- Criar componentes reutilizáveis
- Trabalhar com JSX
- Utilizar Props
- Gerenciar estados com useState
- Executar efeitos com useEffect
- Consumir APIs
- Criar aplicações organizadas
- Desenvolver dashboards profissionais

Componentes e JSX

Componentes

Componentes são blocos reutilizáveis que representam partes da interface da aplicação. Cada componente possui sua própria responsabilidade e pode ser utilizado diversas vezes.

Exemplo:

```
function Button() {  
  return <button>Salvar</button>;  
}
```

JSX

JSX é uma extensão de sintaxe que permite escrever estruturas semelhantes ao HTML dentro do JavaScript.

Exemplo:

```
const nome = "Felipe";  
  
function App() {  
  return <h1>Olá, {nome}</h1>;  
}
```

O JSX torna o código mais legível e facilita a construção das interfaces.

Props e Estados

Props

Props são informações enviadas de um componente para outro.

```
function Card(props){  
  return <h2>{props.title}</h2>;  
}
```

```
<Card title="React" />
```

Estado (State)

O State representa informações que podem mudar durante a execução da aplicação.

```
const [contador, setContador] = useState(0);
```

Sempre que o estado muda, o React atualiza automaticamente a interface.

Eventos e useEffect

Eventos

Os componentes React respondem aos eventos do usuário.

```
<button onClick={incrementar}>  
Adicionar  
</button>
```

Eventos comuns:

- onClick
- onChange
- onSubmit
- onMouseEnter

UseEffect

O Hook useEffect permite executar efeitos colaterais, como carregar dados de uma API.

```
useEffect(() => {  
  buscarCursos();  
}, []);
```

Renderização Condicional e Listas

Renderização Condicional

O React permite exibir componentes diferentes dependendo de uma condição.

```
{
  logado
    ? <Dashboard />
    : <Login />
}
```

Renderizando Listas

```
cursos.map(curso => (
  <Card
    key={curso.id}
    title={curso.nome}
  />
))
```

A propriedade key ajuda o React a identificar cada elemento da lista.

Organização do Projeto e Boas Práticas

Uma aplicação React costuma ser organizada em pastas.

src/

components/

pages/

hooks/

services/

assets/

contexts/

Boas práticas

- Crie componentes pequenos.
- Evite duplicação de código.
- Utilize nomes claros.
- Separe lógica e interface.
- Utilize Hooks corretamente.
- Organize componentes por responsabilidade.
- Mantenha o código limpo e reutilizável.

Próximo Passo

Agora que você domina os fundamentos do React, estará preparado para aprender React Router, consumo de APIs com Fetch e Axios, Context API, autenticação com JWT, gerenciamento de estados, formulários, integração com back-end Node.js e criação de aplicações completas.

Esses conhecimentos formam a base do desenvolvimento Front-end moderno e permitirão construir sistemas profissionais, escaláveis e de alta qualidade.

Essa estrutura fica praticamente **idêntica à apostila de JavaScript**, mudando apenas o conteúdo. Além disso, ela conversa muito bem com o curso "React do Zero ao Dashboard" que você já tem no CourseHub e passa uma imagem consistente para o portfólio.

Hooks Mais Utilizados

Os Hooks são funções especiais introduzidas no React para permitir que componentes funcionais utilizem recursos como estado, ciclo de vida e compartilhamento de lógica.

Os dois Hooks mais utilizados são o **useState** e o **useEffect**, mas existem diversos outros que serão explorados em projetos mais avançados.

useState

O `useState` é utilizado para armazenar informações que podem mudar ao longo da execução da aplicação.

Exemplo:

```
import { useState } from "react";

function Contador() {
  const [contador, setContador] = useState(0);

  return (
    <>
      <h2>{contador}</h2>

      <button onClick={() => setContador(contador + 1)}>
        Incrementar
      </button>
    </>
  );
}
```

Sempre que a função `setContador()` é executada, o React atualiza automaticamente a interface.

useEffect

O Hook `useEffect` permite executar ações quando o componente é renderizado.

É muito utilizado para:

- Buscar informações de uma API;
- Atualizar o título da página;
- Configurar eventos;
- Trabalhar com temporizadores.

Exemplo:

```
useEffect(() => {  
  console.log("Página carregada");  
}, []);
```

O segundo parâmetro ([]) indica que o efeito será executado apenas uma vez, quando o componente for montado.

Consumindo APIs

Grande parte das aplicações modernas obtém informações de servidores.

No React, normalmente utilizamos a função **fetch()** para fazer requisições HTTP.

Exemplo:

```
useEffect(() => {  
  fetch("https://api.exemplo.com/cursos")  
    .then(response => response.json())  
    .then(data => console.log(data));  
}, []);
```

Hoje também é muito comum utilizar bibliotecas como Axios, que simplificam algumas operações.

Após receber os dados da API, normalmente armazenamos essas informações utilizando o useState.

```
const [cursos, setCursos] = useState([]);
```

```
useEffect(() => {  
  fetch("/api/cursos")  
    .then(res => res.json())  
    .then(data => setCursos(data));  
}, []);
```

Essa combinação entre useState e useEffect está presente na maioria dos projetos React profissionais.

React Router

Aplicações React normalmente possuem diversas páginas.

Para navegar entre elas sem recarregar toda a aplicação utilizamos o **React Router**.

Ele permite criar rotas como:

- Home
- Login
- Dashboard
- Perfil
- Cursos
- Configurações

Exemplo:

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
```

Depois definimos as páginas:

```
<Routes>
  <Route path="/" element={<Home />} />

  <Route path="/dashboard" element={<Dashboard />} />

  <Route path="/login" element={<Login />} />
</Routes>
```

Para navegar entre páginas utilizamos o componente Link.

```
<Link to="/dashboard">
  Dashboard
</Link>
```

Essa navegação acontece instantaneamente, sem recarregar toda a aplicação.

Estrutura de um Projeto React

À medida que a aplicação cresce, torna-se importante organizar os arquivos corretamente.

Uma estrutura bastante utilizada é:

src/

assets/

components/

contexts/

hooks/

layouts/

pages/

services/

utils/

App.jsx

main.jsx

Cada pasta possui uma responsabilidade específica.

assets

Armazena imagens, ícones e arquivos estáticos.

components

Contém componentes reutilizáveis, como botões, cards e formulários.

pages

Representa as páginas da aplicação.

services

Centraliza chamadas para APIs.

hooks

Armazena Hooks personalizados.

contexts

Compartilha informações globais utilizando a Context API.

Uma boa organização facilita a manutenção do projeto e melhora o trabalho em equipe.

Conclusão

O React tornou-se uma das tecnologias mais importantes do desenvolvimento Front-end moderno graças à sua simplicidade, desempenho e enorme ecossistema.

Ao longo deste curso você aprendeu os principais conceitos da biblioteca, como componentes, JSX, Props, Estado, Hooks, eventos, renderização condicional, consumo de APIs e organização de projetos.

Esses conhecimentos formam a base para aplicações mais avançadas, incluindo sistemas completos com autenticação, dashboards administrativos, plataformas educacionais, e-commerces e aplicações integradas com back-end.

Continue praticando, criando projetos reais e explorando novas bibliotecas do ecossistema React. A experiência prática é o caminho mais eficiente para se tornar um desenvolvedor Front-end profissional.